PROJECT SUBMISSION

REALIZATION OF FULL ADDER USING A 3-TO-8 DECODER AND BASIC LOGIC GATES

CSC311

Name	Satyaki Banerjee
Roll	CSE/25082/1482
Registration No.	1482
Group No.	6

Submitted for Academic Evaluation

2026

INTRODUCTION

This project implements a 1-bit full adder in Verilog using a 3-to-8 decoder and basic OR logic. The three input combinations are decoded into eight mutually exclusive minterms. Selected decoder outputs are then combined to generate the Sum and Carry outputs.

AIM

To design and verify a full adder using a 3-to-8 decoder, realize the Sum and Carry functions from decoder outputs, and validate all eight input combinations using a Verilog testbench.

OBJECTIVES

• Design a 3-to-8 decoder in Verilog.
• Implement Full Adder Sum and Carry using decoder minterms.
• Develop a testbench covering all eight input combinations.
• Verify the functional outputs through simulation.

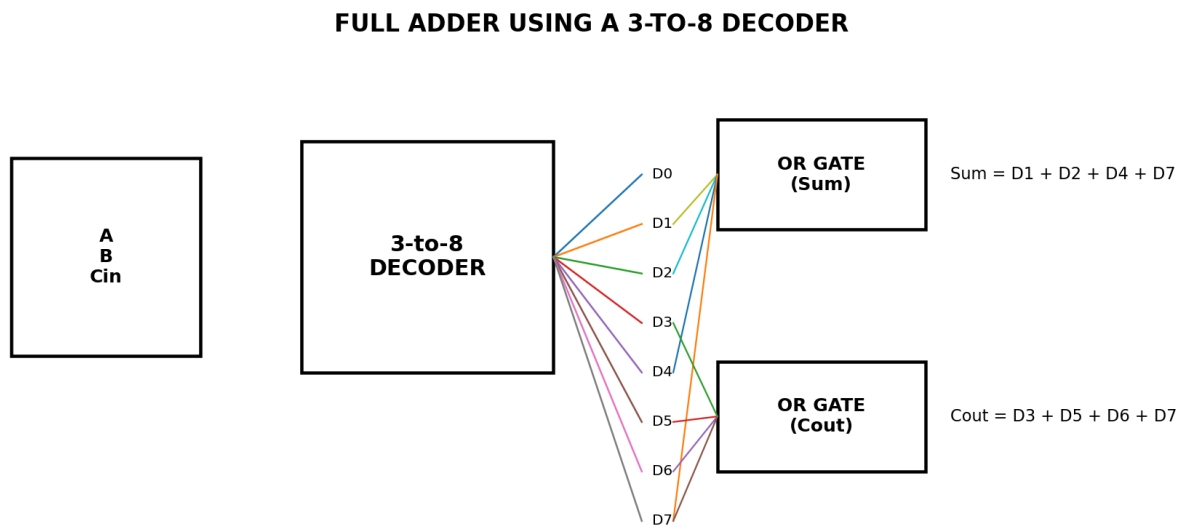


Figure 1: Functional block diagram of the decoder-based full adder.

DESIGN AND LOGIC

3-to-8 Decoder

A 3-to-8 decoder accepts three binary inputs (A, B, Cin) and activates exactly one of its eight outputs D0–D7 for each input combination.

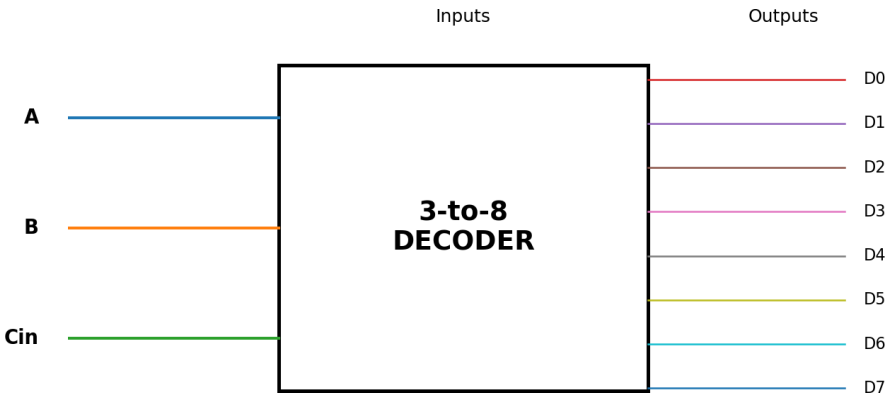


Figure 2: 3-to-8 decoder representation.

Full Adder Equations

Sum = A ⊕ B ⊕ Cin
Cout = AB + BCin + ACin

Using decoder minterms, the Verilog implementation realizes:

Sum = D1 + D2 + D4 + D7
Cout = D3 + D5 + D6 + D7

TRUTH TABLE AND MINTERM MAPPING

A	B	Cin	D0	D1	D2	D3	D4	D5	D6	D7	Sum	Cout
0	0	0	1	0	0	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	1	0	0	0	0	0	1	0
0	1	1	0	0	0	1	0	0	0	0	0	1
1	0	0	0	0	0	0	1	0	0	0	1	0
1	0	1	0	0	0	0	0	1	0	0	0	1
1	1	0	0	0	0	0	0	0	1	0	0	1
1	1	1	0	0	0	0	0	0	0	1	1	1

Table 1: Complete input/output truth table and decoder minterm activation.

For each input combination, only one decoder output is HIGH. The Sum and Carry outputs are formed by OR-ing the required minterms.

VERILOG SOURCE CODE

1. decoder_3to8.v

```
`timescale 1ns / 1ps

module decoder3to8 (
    input wire A,
    input wire B,
    input wire Cin,
    output wire [7:0] D
);

assign D[0] = ~A & ~B & ~Cin;
assign D[1] = ~A & ~B &  Cin;
assign D[2] = ~A &  B & ~Cin;
assign D[3] = ~A &  B &  Cin;
assign D[4] =  A & ~B & ~Cin;
assign D[5] =  A & ~B &  Cin;
assign D[6] =  A &  B & ~Cin;
assign D[7] =  A &  B &  Cin;

endmodule
```

The decoder generates the eight minterms corresponding to all possible combinations of A, B and Cin.

2. full_adder_decoder.v

```
`timescale 1ns / 1ps

module full_adder_decoder (
    input wire A,
    input wire B,
    input wire Cin,
    output wire Sum,
    output wire Cout
);

wire [7:0] D;

decoder3to8 DEC (
    .A(A),
    .B(B),
    .Cin(Cin),
    .D(D)
);

assign Sum  = D[1] | D[2] | D[4] | D[7];
assign Cout = D[3] | D[5] | D[6] | D[7];

endmodule
```

3. full_adder_decoder_tb.v

```
`timescale 1ns / 1ps

module full_adder_decoder_tb;

reg A, B, Cin;
wire Sum, Cout;

full_adder_decoder DUT (
    .A(A), .B(B), .Cin(Cin),
    .Sum(Sum), .Cout(Cout)
);

initial begin
    A=0; B=0; Cin=0; #10;
    A=0; B=0; Cin=1; #10;
    A=0; B=1; Cin=0; #10;
    A=0; B=1; Cin=1; #10;
    A=1; B=0; Cin=0; #10;
    A=1; B=0; Cin=1; #10;
    A=1; B=1; Cin=0; #10;
    A=1; B=1; Cin=1; #10;
    $finish;
end

endmodule
```

SIMULATION / VERIFICATION RESULT

The testbench exercises every possible combination of A, B and Cin. The expected Sum and Cout values match the standard full-adder truth table.

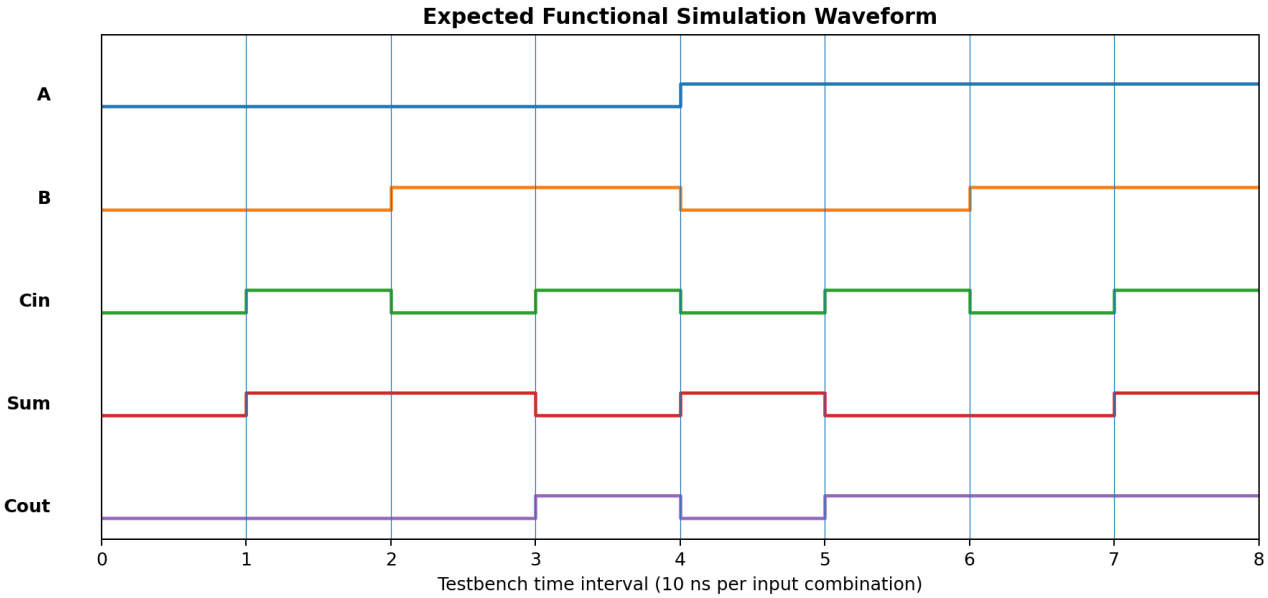


Figure 3: Expected functional waveform corresponding to the eight testbench cases.

Time	A	B	Cin	Sum	Cout
0–10 ns	0	0	0	0	0
10–20 ns	0	0	1	1	0
20–30 ns	0	1	0	1	0
30–40 ns	0	1	1	0	1
40–50 ns	1	0	0	1	0
50–60 ns	1	0	1	0	1
60–70 ns	1	1	0	0	1
70–80 ns	1	1	1	1	1

Table 2: Testbench input sequence and expected full-adder outputs.

RESULT AND CONCLUSION

RESULT

The Verilog design realizes a 1-bit full adder using a 3-to-8 decoder and OR logic. All eight possible input combinations are included in the testbench, and the resulting Sum and Cout values correspond to the required full-adder behavior.

CONCLUSION

The project demonstrates how combinational logic can be implemented from decoder minterms. The modular design separates decoding from output generation, making the circuit easy to understand, verify and extend. The exhaustive testbench provides a simple functional verification method.

Key Components	Purpose
3-to-8 Decoder	Generates D0–D7 minterms
OR Logic	Generates Sum and Cout
Verilog	Hardware description and implementation
Testbench	Checks all eight input combinations

— END OF PROJECT REPORT —